

Fiori Elements 開發課程 12 (期末整合) : Composition App——子表格導覽

環境：BTP ABAP Environment Trial (套件 **ZRAPCLOUD**)，沿用 fe01 ~ fe06 / fe08 ~ fe11 的主線環境
(fe07 的 On-Premise 是唯一例外，這一課不受影響)

Lecture

這一課要解決的問題

fe01 ~ fe11 用過的所有物件都是**單一實體** (**ZI_RC05_NOTE** / **ZI_RC01_TASK** / **ZI_RC01_TASK_SUMMARY**) —— List Report 一張表格、Object Page 一份表單，沒有任何「父子關聯」的畫面。但真實業務資料常常是**主從結構** (訂單 + 明細、Header + Item)，畫面上要能「點開一張訂單，同一頁下方直接看到它的所有明細」，這是 Fiori Elements 最典型、也是最有代表性的畫面模式。

RAP Cloud 課程 rc06 已經建好一組 Composition (Header-Item) 的後端物件——**ZI_RC06_ORDER** (Header，含 **composition [0..*] of ZI_RC06_ORDER_I as _Item**) + **ZI_RC06_ORDER_I** (Item，Dependent Child)，但 rc06 從頭到尾只用 **ABAP Unit 測試驗證過邏輯**，從未曝露成任何 Service，這門課從 fe08 起也一直沒有用過它。這一課要把它做成真正的 Fiori App，畫面上要能看到「訂單列表 → 點進去 → Object Page 下方有一張明細子表格」，這是這門課第一次出現「List Report → Object Page → 內嵌子表格」這種三層畫面結構，也是整個課程的期末整合：串起 Composition (rc06)、兩層 Interface/Projection 架構 (fe08)、Metadata Extension (fe03/fe07/fe08)、Service Definition/Binding (fe08 起) 全部主題。

逐行解說：**as projection on** 跟 **redirected to** 到底在做什麼

這兩個關鍵字是這一課最核心、也最容易看不懂的語法，先用白話講清楚概念，再看程式碼會容易很多。

as projection on ZI_RC06_ORDER 在做什麼：

```
define root view entity ZC_RC06_ORDER
  as projection on ZI_RC06_ORDER
```

這句話的意思是：**ZC_RC06_ORDER (Projection)** 不是自己重新 **select from** 一張資料庫表，而是直接「投影」自 **ZI_RC06_ORDER (Interface)** ——Interface 已經 **select from zrc06_order** 定義好底層資料來源，Projection 只是站在 Interface 上面，決定「Interface 的哪些欄位要曝露給外部消費者、要不要重新命名、要不要加 UI 標記」。這正是 fe08 教過的兩層架構：**Interface** 負責跟資料庫打交道 + 跟 **BDEF** 綁定，**Projection** 負責面對外部 (**Service / UI**)。花括號 { } 裡列的 **order_id / description / created_at / created_by** 都是直接從 **ZI_RC06_ORDER** 繼承過來的既有欄位，沒有任何新邏輯。

_Item : redirected to composition child ZC_RC06_ORDER_I 在做什麼——這是這一課真正新增的觀念，fe08 的單一實體沒有遇過：

Interface 層的 **ZI_RC06_ORDER** 本來就有一個 Composition 關聯 **_Item**，指向 Interface 層的子實體 **ZI_RC06_ORDER_I**：

```
Interface 層：    ZI_RC06_ORDER  --composition _Item-->  ZI_RC06_ORDER_I
```

如果 Projection View 完全不處理這個 `_Item`，它會原封不動被繼承下來，繼續指向 Interface 層的 `ZI_RC06_ORDER_I`——問題是 `ZI_RC06_ORDER_I` 沒有 UI Annotation、沒有面向外部消費者的設計，一旦透過這個關聯導覽過去，等於讓外部消費者「穿透」Projection 層、直接摸到 Interface 層，兩層架構的「面對外部只走 Projection」這個保護就破功了。

`redirected to composition child ZC_RC06_ORDER_I` 這句話做的事，就是把 `_Item` 這個關聯的目的地「改指」到 Projection 層自己的子實體 `ZC_RC06_ORDER_I`，而不是繼續指向 Interface 層的 `ZI_RC06_ORDER_I`：

```
Projection 層:  ZC_RC06_ORDER  --_Item : redirected to composition child-->
ZC_RC06_ORDER_I
```

`composition child` 這個關鍵字額外保留了「這是一個 Composition (不是普通關聯)」的特性 (母子存亡與共、鎖定機制跟著走)，只是單純的 `redirected to` (不加 `composition child`) 會把它降級成普通關聯，失去 Composition 的特殊行為，所以這裡一定要用完整的 `redirected to composition child`。

子實體那邊的 `_Header : redirected to parent ZC_RC06_ORDER` 是完全對稱的邏輯：`ZI_RC06_ORDER_I` 原本的 to-parent 關聯 `_Header` 指向 Interface 層的 `ZI_RC06_ORDER`，同樣要「改指」回 Projection 層的 `ZC_RC06_ORDER`，才能讓子實體的 Projection 也完全留在 Projection 這一層裡，不會反過來又摸回 Interface 層。`redirected to parent` (而非普通 `redirected to`) 同樣是為了保留「這是 to-parent 關聯」的特性。

一句話總結：`as projection on` 決定「這個 Projection View 站在哪個 Interface View 上面」；`redirected to composition child / redirected to parent` 負責把 Composition 關聯的兩端都從 Interface 層改接到 Projection 層，讓整棵 Composition Tree 在 Projection 這一層是完整、封閉、不會漏接回 Interface 層的——這也是官方文件強調「子實體一定要先有自己的 Projection View」的原因：沒有 `ZC_RC06_ORDER_I` 存在，`redirected to composition child ZC_RC06_ORDER_I` 這句話根本沒有東西可以指。

查證：Composition 子表格要用 `@UI.facet` 的 `#LINEITEM_REFERENCE + targetElement`

查證官方文件確認畫面上「Object Page 內嵌一張關聯實體的表格」這個效果，靠的是 `@UI.facet` 這個 Annotation 的特定組合，不是另外某種特殊語法：

- **Adding UI Metadata for UI Consumption** (ABAP_Cloud 官方文件) 示範的 Carrier / Airline 情境，Facet 寫法：`{ purpose: #STANDARD, type: #LINEITEM_REFERENCE, label: '...', position: 10, targetElement: '_Airline' }`——`type: #LINEITEM_REFERENCE` 代表「這個 Facet 是一張表格」，`targetElement` 指向 CDS View 裡實際的 Association / Composition 別名。
- **Adding the Extension Node to the Data Model** (同一份官方文件的另一段) 用 Agency / Review 的 Composition 情境完整示範一樣的組合：`targetElement: '/DMO/ZZ_ReviewZAG'` (那個範例的 Composition 別名)。
- 兩份文件交叉印證：`@UI.facet` 的 `targetElement` 直接對應 CDS View 裡宣告的 **Composition / Association** 別名 (我們這裡就是 `_Item`)，不需要額外宣告什麼「子表格專用」的物件。

查證：Projection 層的 Composition 要用 `REDIRECTED TO COMPOSITION CHILD / REDIRECTED TO PARENT`

fe08 已經教過兩層 Interface/Projection 架構，但當時 `ZI_RC01_TASK` 是單一實體，沒有 Composition。查證官方 ABAP 語言文件 `ABENCDS_PV_ASSOC_REDIRECTED` 確認：**Projection View 要把 Interface 層的 Composition / Association「移到投影層」**，要用 `REDIRECTED TO COMPOSITION CHILD proj_view` (根實

體這邊) 跟 **REDIRECTED TO PARENT proj_view** (子實體這邊) ——這是 Interface 有 Composition 時, 兩層架構唯一合法的接法 (官方文件明講: **REDIRECTED TO COMPOSITION CHILD** 的目標「must be the projection of the original target of the association」, 也就是子實體的 Interface 一定要先有自己的 Projection View, 兩層架構這裡不能省略任何一層)。

Projection BDEF 的寫法也查證了 openSAP 官方 RAP 課程 ([abap-platform-rap-opensap](#), Week 3 Unit 3, Travel / Booking 情境) 現行教材, 確認 Composition 在 Projection BDEF 裡用 **use association _Booking { create; }** (根) / **use association _Travel;** (子) 表示——跟 fe08 已經學過的 **use create; / use action markDone;** 是同一套 **use** 關鍵字家族。

完整程式碼

Projection View (根, **ZC_RC06_ORDER) :**

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Metadata.allowExtensions: true
@EndUserText.label: 'RC06 Order Projection View'
define root view entity ZC_RC06_ORDER
  as projection on ZI_RC06_ORDER
{
  key order_id,
  description,
  created_at,
  created_by,

  _Item : redirected to composition child ZC_RC06_ORDER_I
}
```

Projection View (子, **ZC_RC06_ORDER_I) :**

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Metadata.allowExtensions: true
@EndUserText.label: 'RC06 Order Item Projection View'
define view entity ZC_RC06_ORDER_I
  as projection on ZI_RC06_ORDER_I
{
  key order_id,
  key item_id,
  material_desc,
  quantity,

  _Header : redirected to parent ZC_RC06_ORDER
}
```

Projection BDEF (單一檔案, 綁在根實體 **ZC_RC06_ORDER**, 跟 Interface 一樣一份檔案含兩個 **define behavior for** 區塊) :

```

projection;
strict ( 2 );

define behavior for ZC_RC06_ORDER alias Order
{
    use create;
    use update;
    use delete;

    use association _Item { create; }
}

define behavior for ZC_RC06_ORDER_I alias OrderItem
{
    use update;
    use delete;

    use association _Header;
}

```

⚠ ⚠ **ZC_RC06_ORDER_I** 不需要（也不能）自己建一個獨立的 **Behavior Definition**——這是很容易誤會的地方，值得說清楚原因：

Behavior Definition 綁定的單位是「整棵 **Composition Tree**」，不是「單一 **CDS View**」。一個 RAP Business Object（不管是 Interface 層還是 Projection 層）從頭到尾只有一份 **BDEF** 檔案，永遠掛在根實體上——**ZI_RC06_ORDER**（Interface）的 BDEF 檔案已經是這樣（你可以回頭看它的原始碼，同一份檔案裡就有 `define behavior for ZI_RC06_ORDER alias Header` 跟 `define behavior for ZI_RC06_ORDER_I alias Item` 兩個區塊），Projection 層完全比照同一套規則：子實體的行為，是用「同一份 **BDEF** 檔案裡的第二個 `define behavior for` 區塊」表示，不是另開一個新檔案。

這也解釋了步驟 4 精靈的行為：對 **ZC_RC06_ORDER**（根）執行 **New Behavior Definition**，精靈才會一次生成兩個實體的完整骨架；如果你嘗試對 **ZC_RC06_ORDER_I**（子）右鍵找 **New Behavior Definition**，要嘛選單裡沒有這個選項、要嘛硬做會報錯——因為子實體不是 **Composition Tree** 的根，沒有資格單獨擁有一份 BDEF。判斷一個實體要不要建立自己的 **BDEF**，看它是不是 **root view entity**：**ZC_RC06_ORDER** 有 **root** 關鍵字（見上面 Projection View 的程式碼），**ZC_RC06_ORDER_I** 沒有——這正是它不需要獨立 BDEF 的根本原因。

Metadata Extension（根，**ZC_RC06_ORDER**）——重點是 `@UI.facet` 那個 `#LINEITEM_REFERENCE` 條目：

```

@Metadata.layer: #CORE
@UI: {
    headerInfo: {
        typeName: 'Order',
        typeNamePlural: 'Orders',
        title: { type: #STANDARD, value: 'description' }
    }
}

```

```

}
annotate entity ZC_RC06_ORDER with
{
  @UI.facet: [
    { id: 'GeneralInfo', purpose: #STANDARD, type: #IDENTIFICATION_REFERENCE, label:
'General Information', position: 10 },
    { id: 'Items', purpose: #STANDARD, type: #LINEITEM_REFERENCE, label: 'Items',
position: 20, targetElement: '_Item' }
  ]

  @UI.lineItem: [{ position: 10 }]
  @UI.identification: [{ position: 10 }]
  order_id;

  @UI.lineItem: [{ position: 20 }]
  @UI.identification: [{ position: 20 }]
  description;

  @UI.lineItem: [{ position: 30 }]
  created_at;

  @UI.lineItem: [{ position: 40 }]
  created_by;
}

```

Metadata Extension (子·**ZC_RC06_ORDER_I**) ——只需要 **@UI.lineItem**，決定子表格裡顯示哪些欄位、什麼順序 (**order_id** 是父層鍵值，畫面上不需要重複顯示，故意不標注)：

```

@Metadata.layer: #CORE
annotate entity ZC_RC06_ORDER_I with
{
  @UI.lineItem: [{ position: 10 }]
  item_id;

  @UI.lineItem: [{ position: 20 }]
  material_desc;

  @UI.lineItem: [{ position: 30 }]
  quantity;
}

```

⚠ **踩坑記錄：Metadata Extension 裡引用欄位不能寫 key**——這份講義初版兩處 Metadata Extension 都誤寫成 **key order_id;** / **key item_id;**，Eclipse 啟用時報 **Unexpected token "order_id". Expected was ";"** (游標停在 **key** 後面，代表剖析器把 **key** 當成一個獨立陳述式，**order_id** 變成不該出現的下一個 token)。原因：**key** 這個關鍵字只在 CDS View 本體 (**define view entity**) 宣告欄位時使用，用來標記這個欄位是主鍵；但 Metadata Extension (**annotate entity ... with { }**) 不是在宣告欄位，只是引用一個

已經存在的欄位來掛 **Annotation**——這個欄位是不是主鍵，早在底層 CDS View 已經定案，Metadata Extension 這一層完全不需要（也不能）重複宣告，直接寫欄位名稱即可。上面兩段程式碼已經修正為 `order_id; / item_id;`（不帶 `key`），如果你是照著舊版截圖操作，記得把這兩處的 `key` 拿掉。

Service Definition (ZRC06_SD)——根跟子都要曝露（照 openSAP 官方 Travel / Booking 範例的慣例，同一個 Service 一次曝露整個 Composition Tree，不是只曝露根）：

```
define service ZRC06_SD {  
  expose ZC_RC06_ORDER as OrderHeader;  
  expose ZC_RC06_ORDER_I as OrderItem;  
}
```

⚠ 踩坑記錄：**ORDER** 是保留字，不能拿來當曝露的 **Entity** 名稱——原始版本寫 `expose ZC_RC06_ORDER as Order;`，Eclipse 啟用報 `'Order' is a reserved keyword`。原因：**ORDER** 是 SQL / CDS 的保留字（**ORDER BY** 子句用的那個 **ORDER**），`define service` 曝露出去的名稱最終會變成 OData Entity Set 名稱，這一層的命名跟 SQL 關鍵字共用同一份保留字清單，不能直接拿一個保留字當名稱。修法：改一個不衝突的名字即可，這裡改用 `OrderHeader`（跟子實體 `OrderItem` 對稱，語意也更精確——這是 Composition 的根 / 表頭）。只有 **Service Definition** 這裡的曝露名稱會撞到保留字，**BDEF** 裡的 `alias Order`（見上面 **Projection BDEF** 程式碼）是另一個獨立的命名空間，不受影響，不需要跟著改——這是這一課第二個因為命名撞到系統保留字/語法規則而出的錯，跟上一個 `key` 的錯誤一樣，都是純語法層級的坑，不影響你已經理解的 Composition / Projection 概念本身。

Eclipse ADT 建立步驟（套件 **ZRAPCLOUD**）

1. **Projection View (根)**：對套件 **ZRAPCLOUD** 右鍵 → **New** → **Data Definition**，Name 填 `ZC_RC06_ORDER`，Template 選 **Define View**，貼入上面的程式碼，存檔（先不要急著啟用——這時候 `_Item` 指向的 `ZC_RC06_ORDER_I` 還不存在，啟用會報錯，這是預期中的，繼續下一步）
2. **Projection View (子)**：對套件右鍵 → **New** → **Data Definition**，Name 填 `ZC_RC06_ORDER_I`，貼入程式碼，存檔
3. 一起啟用：這兩個 Projection View 互相參照對方（根指向子、子指向根），單獨對其中一個按 **Ctrl+F3** 會因為對方還是 inactive 而失敗——正確做法是用 Eclipse 工具列的 **Activate Inactive ABAP Development Objects** 按鈕（圖示是一疊文件 + 打勾符號，位於工具列偏右側，Debug/Run 按鈕群組附近）。點下去會跳出「Inactive objects for <使用者>」對話框，列出目前這個使用者名下所有 inactive 的物件（不限這一課，其他還沒啟用的物件也會混在一起列出來），勾選這次要一起啟用的 `ZC_RC06_ORDER / ZC_RC06_ORDER_I` 兩個 **Data Definition**（其他不相關的 inactive 物件不要勾），按 **Activate**——系統會自動排序，一次把兩個互相參照的物件正確啟用，不需要糾結先後順序。這個對話框之後每次遇到「兩個以上物件互相參照、必須一起啟用」的情境都可以用（例如下面步驟 4 的 Projection BDEF 如果啟用失敗，也可以用這個工具跟 Projection View 一起選起來啟用）。
4. **Projection BDEF**：對 `ZC_RC06_ORDER` 右鍵 → **New Behavior Definition**，**Implementation Type** 選 **Projection**（⚠ 這裡只能選 Projection，不是隨便挑的——**Managed / Unmanaged** 是留給「直接對資料庫表」的 Interface View 用的，見下方對照表），**Next** → **Finish** 後精靈會自動產生兩個實體（**Order / OrderItem**）的完整骨架，全部操作預設用 **use** 展開（這是官方精靈的標準行為，不是要你從空白開始寫），把骨架內容替換 / 調整成上面的最終版本，存檔→啟用

Implementation Type 對照表（回答「一般 CDS View 建 BDEF 要選哪個」）：

| Implementation Type | 用在哪種 CDS View | 誰負責持久化邏輯 | |---|---|---| | **Managed** | Interface View (`as select from <table>` , 直接對資料庫表) | 框架自動生成 INSERT/UPDATE/DELETE , 只要宣告 `create;update;delete;`——rc06 的 `ZI_RC06_ORDER` 就是這樣建的 (`managed implementation in class zbp_i_rc06_order unique;`) , 這門課到目前為止所有 Interface 層 BDEF 都選這個 || **Unmanaged** | Interface View (同上) | 完全自己寫 (Local Types Include 手動下 SQL) , 適合複雜業務邏輯或整合既有 Function Module/BAPI , RAP 課程已教過 || **Projection** | Projection View (`as projection on <Interface>` , 投影自另一個已有 BDEF 的 Interface View) | **不處理持久化** , 只是重新曝露 / 裁剪 Interface 層已有的操作 , 實際資料庫動作委託給 Interface 層執行——這一課唯一合法的選項 || **Abstract** | CDS Abstract Entity (沒有底層資料來源 , 純型別容器) | 不適用 , 這門課沒用過 |

1. **Metadata Extension (根)** : 對 `ZC_RC06_ORDER` 右鍵 → **New Metadata Extension** , Name 同名 , 貼入程式碼 , 存檔→啟用
2. **Metadata Extension (子)** : 對 `ZC_RC06_ORDER_I` 右鍵 → **New Metadata Extension** , Name 同名 , 貼入程式碼 , 存檔→啟用
3. **Service Definition** : 對套件右鍵 → **New** → **Service Definition** , Name 填 `ZRC06_SD` , 貼入程式碼 , 存檔→啟用
4. **Service Binding** : 對 `ZRC06_SD` 右鍵 → **New Service Binding** , Name 填 `ZRC06_SB` , Binding Type 選 **OData V4 - UI** , 選 Transport Request → Finish 。 ⚠️ ⚠️ 建立完的 **Service Binding** 預設是 **inactive** , 一定要先 **Ctrl+F3** 啟用 , **Publish** 按鈕才會真的生效——容易誤會「按 Publish」本身就會順便啟用 , 實際上「建立→啟用→Publish」是三個獨立步驟 , 順序不能顛倒也不能省略 : 如果物件還是 inactive 狀態就直接按 Publish , 可能整個沒反應、或是 Publish 出去的是舊的 / 不完整的版本 , 看起來像「有做但沒生效」 , 很容易誤判成別的問題 (fe08 起每一課都遇過同一個坑 , 這裡再強調一次) 。
5. ⚠️ ⚠️ 補測試資料 : `ZI_RC06_ORDER` 底層的表格 `zrc06_order` 目前是空的 (rc06 建立這組物件時只用 ABAP Unit 測試驗證邏輯 , 測試類別 `ZCL_RC06_ORDER_TEST` 的 `setup()` 方法每次執行前都會先 **DELETE** 測試資料 , 所以沒有留下任何持久資料) ——如果現在直接去前端按 Go , 會查到 0 筆 , 這不是設定錯誤 , 是資料庫本來就沒有資料 。 **ABAP Cloud** 不允許建立傳統 **Program (Report)** , 要用實作 **IF_OO_ADT_CLASSRUN** 介面的 **Class** 才能在 **ADT** 裡直接執行 : 對套件 **ZRAPCLOUD** 右鍵 → **New** → **ABAP Class** , Name 填 `ZCL_RC06_SEED_DATA` , 貼入 :

```
``abap CLASS zcl_rc06_seed_data DEFINITION PUBLIC FINAL CREATE PUBLIC.
```

```
PUBLIC SECTION. INTERFACES if_oo_adt_classrun.
```

```
ENDCLASS.
```

```
CLASS zcl_rc06_seed_data IMPLEMENTATION.
```

```
METHOD if_oo_adt_classrun~main.
```

```
MODIFY ENTITIES OF zi_rc06_order ENTITY Header CREATE FIELDS ( order_id description ) WITH VALUE #( ( %cid = 'H1' order_id = 'RC06DEMO01' description = 'FE12 Demo Order' ) )
```

```
ENTITY Header CREATE BY \Item FIELDS ( item_id material_desc quantity ) WITH VALUE #( ( %cid_ref = 'H1' %target = VALUE #( ( %cid = 'I1' item_id = '0010' material_desc = 'Widget A' quantity = '5' ) ( %cid = 'I2' item_id = '0020' material_desc = 'Widget B' quantity = '3' ) ) ) )
```

```
FAILED DATA(ls_failed) REPORTED DATA(ls_reported).
```

COMMIT ENTITIES RESPONSE OF zi_rc06_order FAILED DATA(ls_failed_commit) REPORTED
DATA(ls_reported_commit).

IF ls_failed-header IS NOT INITIAL OR ls_failed-item IS NOT INITIAL OR ls_failed_commit-header IS NOT
INITIAL. out->write('FAILED - check ls_failed / ls_failed_commit'). ELSE. out->write('OK - Created
RC06DEMO01 with 2 items'). ENDIF.

ENDMETHOD.

ENDCLASS. ``

這段 EML 語法直接照抄 rc06 既有測試類別 **ZCL_RC06_ORDER_TEST** 已經驗證過能動的寫法（拿掉了它的
cleanup 邏輯），一次 Create-by-Association 建立一筆 Header（**RC06DEMO01**）+ 兩筆 Item。存檔 → 啟用 →
點編輯器工具列的綠色 **Run** 按鈕（或右鍵 → **Run As** → **Application**），輸出顯示在 **Console** 分頁，看到 **OK**
- **Created RC06DEMO01 with 2 items** 才算成功，才能繼續下一步。

⚠ 這一課刻意沒有加 **with draft;**——跟 fe08 的發現一致，留給動手練習

ZI_RC06_ORDER 的 Interface BDEF（rc06 建立）只有 **create; update; delete;**，沒有 **with draft;**。呼應
fe08 已經驗證過的結論——「**Draft 是 Fiori Elements Create/Edit 按鈕的前提**」——這一課的 Object Page 預
期會顯示 Items 子表格（讀取沒問題），但不會出現 **Create / Edit** 按鈕，Header 跟 Item 都一樣。這不是這一
課的錯誤，是刻意保留的教學缺口：**動手練習第 1 題會請你回頭幫 ZI_RC06_ORDER 加上 with draft;**，這是
整門課最後一次把 rc05（Draft）、rc06（Composition）、fe08（Interface/Projection）三個主題真正串在一起的
機會。

⚠ 更精確的規則（使用者實測補上的細節）：**Delete** 按鈕不受 **Draft** 缺席影響，正常出現（畫面右上角會看
到 **Delete** 按鈕，灰色、勾選列之後才會亮起）——之前 fe08 只確認了「沒有 Create/Edit」，沒有明確講過
Delete；這一課用真實截圖證實 **Draft 只是 Create / Update（Edit）按鈕的前提，不影響 Delete**。這跟 RAP
框架的設計邏輯一致：Draft 管的是「編輯中」這個工作階段的概念（Create/Update 需要先進草稿、確認後才
正式落地），Delete 是一次性的直接動作，語意上不需要草稿流程，所以 Fiori Elements 範本不會因為沒有
Draft 就連 Delete 按鈕都藏起來。

前端：VS Code Fiori Generator Step by Step

沿用 fe01 已經連好的 **TRL** 系統，不用重新輸入 URL / 登入：

1. **Ctrl+Shift+P** → **Fiori: Open Application Generator**
2. Template 選 **List Report Page**
3. Data Source 選 **Connect to a System** → System 選既有的 **TRL**
4. **Service** 搜尋框輸入 **ZRC06_SB**，選 **ZRC06_SB > ZRC06_SD (0001)**
5. 「Download value help metadata」選 **Yes**
6. Main Entity 選 **OrderHeader**（不是 **OrderItem**——**OrderItem** 是透過 Composition 從 Object Page 導
覽進去的子項目，不是獨立的 List Report 對象）。⚠ 較新版本的精靈這裡會多一個 **Navigation**
Entity 欄位（必填，預設值是 **None**）——選 **None** 就好，不要選 **_Item**。原因：這個欄位搭配下面
「Automatically add table columns...」選 Yes 時，是精靈自動幫你生成 **@UI.facet / @UI.lineItem** 這
類 Annotation 到本機專案的 **annotation.xml**；但這一課的 **Items** 子表格 Facet 我們已經自己寫在後端
Metadata Extension 裡（**ZC_RC06_ORDER** 的 DDLX），如果這裡選 **_Item** 讓精靈也生成一份本機版本，
會撞上 fe03 已經驗證過的規則——同一個（**Target, Term, Qualifier**）的 **Annotation**，本機檔案會
整個覆蓋後端版本，不是合併，容易把我們手寫好的 Facet 設定覆蓋掉、畫面跟預期不一致。選 **None** 讓

精靈不要碰這個 Composition，Facet 完全交給後端 Metadata Extension 決定。✅ 已實測驗證：選 **None** 完全正確——Items 子表格正常顯示兩筆明細，沒有任何 Annotation 衝突或畫面異常。Table Type 選 **Responsive**

7. Project Attributes：Module Name 填 **fe12ordercomposition** (⚠️ 全小寫)，Enable TypeScript：**No**，Project Folder Path 指到 **src/ABAP_Training_Fiori_Elements/**，**Finish**
8. 開終端機，先 **cd** 進 **src/ABAP_Training_Fiori_Elements/fe12ordercomposition/**，再執行 **npm start**，等終端機印出 **Server started / URL: http://localhost:XXXX** (Port 看實際印出的號碼，不保證是 8080)
9. 瀏覽器自動開啟該 Port 的 **/test/flp.html#app-preview**；如果卡在瀏覽器原生 Basic Auth 帳密框，把該 Port 的 process 砍掉重開 (fe04 講義記錄過的已知認證異常)
10. 回到瀏覽器按 **Go**，確認查到 Eclipse 步驟 9 建立的 **RC06DEM001** 這筆訂單 (如果查到 0 筆，先確認步驟 9 的 **ZCL_RC06_SEED_DATA** 真的執行成功、Console 有印出 **OK**)

驗證重點 (跟 fe09 的「確認沒有按鈕」剛好相反，這次是「確認子表格有出現」)：

1. List Report 顯示訂單清單 (**order_id / description / created_at / created_by**)，看到 **RC06DEM001** 這筆資料——沒有 **Create** 按鈕，但有 **Delete** 按鈕 (灰色、勾選列才會亮起)。⚠️ 沒有 **Create / Edit** 按鈕是正確的、預期中的結果，不是錯誤：因為 **ZI_RC06_ORDER** 目前還沒有加 **with draft**；呼應 fe08 已經驗證過的結論「Draft 是 Fiori Elements Create/Edit 按鈕的前提」——Delete 不受影響則是這一課才確認的更精確規則 (見上面「更精確的規則」)
2. 點進 **RC06DEM001** 這筆訂單的 Object Page，應該要在 **General Information** 之外看到第二個區塊 **Items**，底下是一張表格，顯示剛才建立的兩筆明細 (**0010 / Widget A / 5** 跟 **0020 / Widget B / 3**)——這張表格就是 **@UI.facet** 的 **#LINEITEM_REFERENCE** 產生的效果，是這一課的核心驗收點
3. 這張子表格同樣沒有 **Create** 按鈕 (同樣是還沒加 **with draft** 的緣故)，**Delete** 一樣正常可用

學習目標

- 能講出 Composition 在 Fiori Elements 畫面上呈現為「Object Page 內嵌子表格」，靠的是 **@UI.facet** 的 **type: #LINEITEM_REFERENCE + targetElement** 指向 Composition 別名
- 能寫出 Projection 層承接 Composition 的語法：根實體 **REDIRECTED TO COMPOSITION CHILD**，子實體 **REDIRECTED TO PARENT**——知道這是官方文件明講「兩層架構遇到 Composition 時唯一合法接法」，子實體一定要有自己的 Projection View，不能省略
- 能寫出 Projection BDEF 對 Composition 的展開語法：**use association _Item { create; }** (根) / **use association _Header;** (子)
- 能講出這一課跟 fe08、fe09、rc05、rc06 的整合關係：Composition (rc06 後端) + 兩層架構 (fe08) + Metadata Extension (fe03/fe07/fe08) + 沒有 Draft 就沒有 Create/Edit (fe08 發現，這一課再次印證)
- 知道 Main Entity 在 Generator 精靈裡選的是 Composition 的根實體，子實體透過畫面內建的 Facet 導覽進去，不是獨立的 List Report

物件清單

套件 **ZRAPCLOUD** (BTP Trial，非 **\$TMP**，需要傳輸請求)：

物件	型別	說明
ZI_RC06_ORDER / ZI_RC06_ORDER_I	DDLS + BDEF (Interface)	rc06 既有物件，這一課沒有修改
ZC_RC06_ORDER	DDLS (Projection View · 根)	as projection on ZI_RC06_ORDER · _Item : redirected to composition child ZC_RC06_ORDER_I
ZC_RC06_ORDER_I	DDLS (Projection View · 子)	as projection on ZI_RC06_ORDER_I · _Header : redirected to parent ZC_RC06_ORDER
ZC_RC06_ORDER	BDEF (Projection)	use create/update/delete + use association _Item { create; } (根) / use update/delete + use association _Header; (子)
ZC_RC06_ORDER	DDLX (Metadata Extension · 根)	@UI.facet 含 #LINEITEM_REFERENCE 指向 _Item
ZC_RC06_ORDER_I	DDLX (Metadata Extension · 子)	子表格欄位的 @UI.lineItem
ZRC06_SD	Service Definition	expose ZC_RC06_ORDER as OrderHeader; expose ZC_RC06_ORDER_I as OrderItem; (Order 是保留字，改用 OrderHeader)
ZRC06_SB	Service Binding (OData V4- UI)	已 Publish
ZCL_RC06_SEED_DATA	Class (IF_OO_ADT_CLASSRUN)	補測試資料用，EML 建立 RC06DEMO01 + 2 筆 Item (ABAP Cloud 不允許傳統 Program，改 用這個介面才能在 ADT 直接 Run)
fe12ordercomposition (前端專案)	本機	VS Code Fiori Generator 產生

動手練習

輪到你了：

1. 主要練習 (整門課的收尾)：回頭幫 ZI_RC06_ORDER 的 Interface BDEF 加上 with draft;，讓 Create / Edit 按鈕真正出現。提示：參考 rc05 講義的 Draft 語法細節 (draft table 緊接在 persistent table 之後、total etag 緊接在 lock master 之後)，你需要新建兩張 Draft Table (ZRC06_ORDER_D / ZRC06_ORDER_I_D)，欄位要跟正式表逐一對應、第一欄位是 abap.clnt、要有 "%admin"：include sych_bdl_draft_admin_inc;。這是這門課最後一次動手練習，刻意留給你自己走一遍「查文件→設計→踩錯→修正」的完整過程，而不是照抄現成答案。
2. 這一課的 @UI.facet 只用了 #IDENTIFICATION_REFERENCE (一般欄位表單) 跟 #LINEITEM_REFERENCE (子表格) 兩種 Facet 類型。查一下官方文件還有哪些 Facet type (提示：

#DATAPOINT_REFERENCE、#CHART_REFERENCE)，想一想如果要在這個 Order 的 Object Page 加一個「本訂單總數量」的 KPI 顯示，該用哪一種

- 目前 ZC_RC06_ORDER_I 的 Metadata Extension 完全沒有 @UI.identification——代表子表格裡的每一列沒有辦法點進去看單獨的 Item Object Page（子項目本身也沒有自己的 Object Page 設定）。想一想：什麼情境下你會想讓子項目也有自己的 Object Page？什麼情境下像這一課一樣，只需要在父層的內嵌表格裡看就夠了？

驗證方式

✅ 已端對端驗證成功（截圖為證）：後端物件依照 Eclipse ADT 建立步驟建立、啟用、Service Binding Publish 成功，ZCL_RC06_SEED_DATA 補上 RC06DEM001 測試資料後，前端用 VS Code Fiori Generator 產生 App、npm start 本機預覽，Object Page 正確顯示 General Information / Items 兩個分頁，Items (2) 子表格正確顯示兩筆明細 (0010 / Widget A / 5.00、0020 / Widget B / 3.00)，畫面右上角沒有 Create 按鈕、有 Delete 按鈕 (Ctrl+Del)，跟這一課「Draft 只是 Create/Update 前提、不影響 Delete」的結論完全一致。

這一課用到的 @UI.facet #LINEITEM_REFERENCE + targetElement 語法、Projection 層的 REDIRECTED TO COMPOSITION CHILD / REDIRECTED TO PARENT 語法、Front-end 精靈 Navigation Entity 選 None 的判斷，全部經過這次實測確認正確——這是這門課少數幾次「先寫講義、後動手驗證」的情況（跟 fe01 ~ fe11 每一課都先實測再寫講義的順序相反），驗證結果證實查證官方文件（ABAP_Cloud UI Annotation 文件×2、ABAP 語言參考 ABENCDS_PV_ASSOC_REDIRECTED、openSAP 官方 RAP 課程 Week 3 Unit 3）後直接寫成講義這個做法在這一課是可靠的，過程中真正出錯的兩處（Metadata Extension 誤寫 key、Service Definition 撞保留字 Order）都是沒有查證來源、純憑印象寫的地方，印證「有查證的語法風險低、沒查證的地方才容易出錯」這個規律。

思考題

- 這一課的 Service Definition 一次曝露了 OrderHeader 跟 OrderItem 兩個實體。如果只曝露 OrderHeader（拿掉 expose ZC_RC06_ORDER_I as OrderItem; 這一行），Object Page 上的 Items 子表格還會不會正常顯示？（提示：想一想 @UI.facet 的 targetElement 走的是 CDS Association 路徑導覽，跟 Service Definition 有沒有獨立曝露這個子實體本身，是不是兩件事）
- fe08 用兩層架構（Interface / Projection）處理單一實體，這一課用兩層架構處理 Composition（兩個實體，各自兩層，變成四個 CDS View）。如果 Composition 有三層（例如訂單→明細→序號），兩層架構會變成幾個 CDS View？這種複雜度增長，你覺得什麼時候會讓兩層架構「不值得」，該考慮別的設計？
- 這一課的 Item 子表格用 #LINEITEM_REFERENCE，欄位可以直接在內嵌表格裡編輯（如果有 Draft 的話）；如果需求是「子項目資料量很大，不適合全部塞進 Object Page，需要獨立一個頁面」，Fiori Elements 有沒有支援這種「子項目自己也是一個完整 List Report/Object Page」的模式？（提示：查一下 Composition 場景裡，子實體自己被獨立 expose ... as OrderItem 這件事，是不是就已經隱含了這個可能性）

答案

見 ZC_RC06_ORDER (DDL / BDEF / DDLX)、ZC_RC06_ORDER_I (DDL / DDLX)、ZRC06_SD、ZRC06_SB、ZCL_RC06_SEED_DATA，皆建立於 BTP Trial 系統套件 ZRAPCLOUD。ZI_RC06_ORDER / ZI_RC06_ORDER_I 沿用 rc06 既有物件未修改。✅ 已端對端驗證成功：fe12ordercomposition 前端專案 npm start 本機預覽，Object Page 的 Items 子表格正確顯示 RC06DEM001 的兩筆明細，畫面行為（無 Create、有 Delete）符合預期。